Project: To-Do List API **CR ID:** CR-003 **Title:** Scheduled Detection of Overdue Tasks **Type:** Enhancement — Automation **Priority:** Medium **Status:** Proposed

---

1. Problem Statement

A task whose due date has passed and which is not yet `DONE` is not flagged anywhere today. Overdue status can only be seen by a client manually comparing a task's `dueDate` against the current date after fetching it — there is no server-side detection or record of overdue tasks.

2. Proposed Change

Introduce a background process that periodically scans tasks and identifies those that are overdue (due date in the past, status not `DONE`). The detection frequency is configurable rather than hard-coded. This CR covers *detection* of overdue tasks; whether that detection also creates a persisted notification record is a scope decision requiring explicit sign-off (see §"Additional Design Consideration" in the project's CR brief, and the recommendation in `TECHNICAL_DESIGN.md`).

3. Scenario

1. Task #17, "Submit vendor contract," has `dueDate = 2026-10-10` and `status = IN_PROGRESS`.
2. The current business date advances to `2026-10-15` without the task being updated.
3. On its next scheduled run, the overdue-detection job scans all tasks whose `dueDate` is before today and whose `status` is not `DONE`.
4. Task #17 matches. The job logs that it is overdue (and, if the notification option is approved, creates a single notification record for it — see CR-003's design options).
5. If the job runs again later the same day (or any day) before Task #17's status or due date changes, it recognizes the task was already flagged and does not log or notify about it a second time.

4. Functional Requirements

- **FR-01:** The system shall run a scheduled job on a configurable interval (e.g. via a cron expression in `application.yml`), not hard-coded in Java.
- **FR-02:** On each run, the job shall identify all tasks where `dueDate` is earlier than the current business date **and** `status` is not `DONE`.
- **FR-03:** The job shall not modify the `status` or any other field of the tasks it detects — detection is read-only with respect to the `Task` table.
- **FR-04:** Each run shall log the number of overdue tasks found (and, at minimum, their ids) for operational visibility.
- **FR-05:** The job's core detection logic shall be independently testable without waiting for or triggering the actual cron schedule (i.e. callable directly in a test).
- **FR-06:** A task already identified as overdue in a previous run shall not be flagged/logged/notified again on a subsequent run unless its due date or status changes and it becomes overdue again under new circumstances (if the notification option, below, is approved).
- **FR-07 (conditional on scope decision):** If notification creation is approved, the job shall create at most one notification record per currently-overdue task, not one per scheduled run.

5. Acceptance Criteria

- Running the job against a dataset containing both overdue and non-overdue tasks identifies exactly the overdue ones (due date in the past, not `DONE`), with no false positives or negatives.
- A task due today is **not** considered overdue (only strictly past due dates qualify, per the business rule).
- A task that is `DONE`, regardless of due date, is never considered overdue.
- The job's schedule frequency can be changed via configuration without a code change.
- Running the detection logic directly (outside the scheduler trigger) produces the same result as waiting for the scheduled trigger.

6. Scope

In Scope:

- Scheduled, configurable-interval detection of overdue tasks.
- Structured logging of detection results.
- A documented, explicit recommendation on whether detection should also create notification records (see `TECHNICAL_DESIGN.md` §"Additional Design Consideration").

Out of Scope:

- Automatically changing a task's `status` because it became overdue (the project brief explicitly prohibits modifying the existing status model as part of this CR).
- Any outbound notification channel (email, push, SMS) — if notification records are approved, they are persisted rows only, not delivered messages. Delivery is a separate, future concern.
- A user/authentication system — none currently exists in this project, and this CR does not introduce one.

7. Approval

- **Business Approval:** Pending
- **Technical Approval:** Pending
- **Implementation Status:** Not Started